

Visão Geral do ADO.NET

ADO.NET (Active Data Objects for .NET) é um conjunto de classes fornecidas pela plataforma .NET para obter e gerir dados armazenados em fontes de dados como bases de dados relacionais (SQL Server, MySQL, Oracle), ficheiros XML, folhas de cálculo e até serviços de dados.

É a principal tecnologia de acesso a dados no .NET, projetada para trabalhar em dois modos:

- **Connected (conectado):** Mantém uma conexão direta e contínua com a fonte de dados.
- **Disconnected (desconectado):** Trabalha com dados em memória local (como DataSet e DataTable), sem manter a conexão ativa com a base de dados.

A arquitetura do ADO.NET é baseada em dois componentes principais:

1 - Data Providers (Provedores de Dados) e 2 - Modelo Desconectado.

1. Data Providers (Provedores de Dados)

Os provedores de dados são bibliotecas específicas que gerem a interação entre a aplicação e a base de dados. Eles permitem conexões, execução de comandos SQL e leitura de dados.

Os provedores mais comuns são:

- System.Data.SqlClient: Para SQL Server.
- System.Data.OleDb: Para fontes de dados acedidas via OLE DB.
- System.Data.OracleClient: Para Oracle (obsoleto, substituído por Oracle.ManagedDataAccess).
- System.Data.SQLite: Para SQLite.
- System.Data.Common: Uma interface genérica para trabalhar com diferentes fontes.

Componentes de um Provedor de Dados:

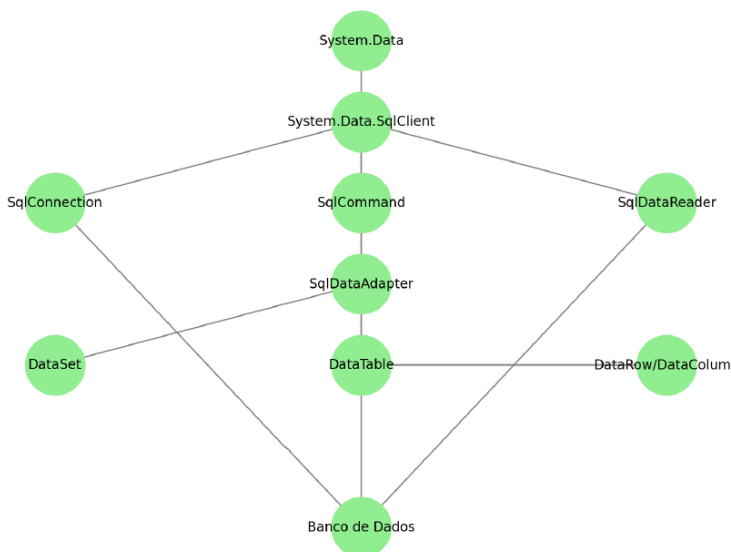
- **Connection:** Representa a conexão física com a fonte de dados. Ex: SqlConnection, OleDbConnection.
- **Command:** Representa uma instrução SQL ou chamada de procedimento armazenado que pode ser executada na fonte de dados. Ex: SqlCommand.
- **DataReader:** Fornece acesso somente de leitura e apenas acesso sequencial aos dados obtidos. Ex: SqlDataReader.
- **DataAdapter:** Atua como ponte entre a base de dados e o DataSet num modelo desconectado.

2. Modelo Desconectado

O modelo desconectado permite que os dados sejam obtidos da base de dados e manipulados em memória, sem a necessidade de manter uma conexão aberta. Os componentes principais do modelo desconectado são:

- **DataSet:** Uma coleção de tabelas (DataTable), relações e restrições. Usado para manipular dados em memória.
- **DataTable:** Representa uma tabela de dados em memória. Pode ser usado independentemente ou dentro de um DataSet.
- **DataRow e DataColumn:** Representam, respectivamente, linhas e colunas de uma tabela.

Diagrama das Classes de ADO.NET no .NET Framework



Funcionamento Geral da Arquitetura

Conectado (Connected): A aplicação abre uma conexão com a base de dados usando Connection. Executa comandos SQL com Command e obtém os dados com DataReader para processá-los imediatamente. Melhor para operações rápidas e transações curtas. Requer conexão constante.

Desconectado (Disconnected): O DataAdapter preenche um DataSet com os dados. O DataSet é manipulado localmente, e as mudanças são enviadas para a base de dados quando necessário. Ideal para cenários onde uma conexão contínua não é prática. Boa para aplicações distribuídas ou offline.

Vantagens do ADO.NET

- **Eficiência:** Modelo conectado permite operações rápidas e diretas. Modelo desconectado reduz a carga no servidor.
- **Flexibilidade:** Suporte a múltiplas fontes de dados. Trabalha bem em cenários online e offline.

- Segurança: Proteção contra SQL Injection com parâmetros. Controle explícito de conexões e transações.

Funcionamento do SqlDataAdapter

O SqlDataAdapter é projetado para funcionar no modo desconectado. Ele gere a conexão com a base de dados automaticamente durante as operações de:

Fill: Preencher o DataSet ou DataTable com os dados da base de dados.

Update: Aplicar as alterações do DataSet ou DataTable à base de dados.

Quando se chama adapter.Fill(dataSet, "Clientes") ou adapter.Update(dataSet, "Clientes"):

- O SqlDataAdapter abre a conexão se ela estiver fechada, executa o comando SQL associado (SELECT, INSERT, UPDATE, DELETE), fecha a conexão automaticamente após a operação.

Por que é que connection.Open() não é necessário?

Não é necessário chamar connection.Open() manualmente porque:

- Conexão Automática: O SqlDataAdapter verifica o estado da conexão, se a conexão estiver fechada, o SqlDataAdapter abre-a antes de executar a operação, após a operação, o SqlDataAdapter fecha a conexão.

Segurança e Eficiência: reduz a possibilidade de esquecer de fechar a conexão, prevenindo vazamentos de recursos. O SqlDataAdapter garante que a conexão é mantida aberta apenas pelo tempo necessário.

Quando seria necessário usar connection.Open()?

- Reutilizar a mesma conexão em várias operações diferentes.
- Executar comandos diretamente usando objetos como SqlCommand fora do escopo do SqlDataAdapter.

Para que serve o SqlCommandBuilder?

O SqlCommandBuilder elimina a necessidade de escrever manualmente os comandos SQL para operações de modificação (inserção, atualização e exclusão). Ele analisa a consulta SQL associada ao SqlDataAdapter (geralmente um comando SELECT) e gera automaticamente os comandos SQL apropriados com base na estrutura da tabela.

- Gera automaticamente os comandos INSERT, UPDATE e DELETE para o SqlDataAdapter, com base na consulta SELECT fornecida ao SqlDataAdapter,
- Após gerar os comandos, ele os associa automaticamente às propriedades InsertCommand, UpdateCommand e DeleteCommand do SqlDataAdapter.

Quando é necessário o SqlCommandBuilder?

- Quando se usa o método Update do SqlDataAdapter para sincronizar as alterações feitas no DataSet ou DataTable com a base de dados.
- Quando não se criam manualmente os comandos SQL INSERT, UPDATE e DELETE.

Quando não é necessário o SqlCommandBuilder?

- Existem os comandos SQL definidos manualmente.
- As tabelas não têm uma chave primária: O SqlCommandBuilder depende da presença de uma chave primária na tabela para gerar os comandos corretamente. Se a tabela não tiver uma chave primária, devem configurar os comandos manualmente.

Notas sobre o SqlCommandBuilder

- Performance: O SqlCommandBuilder gera comandos SQL dinamicamente cada vez que o Update é chamado.
- Consulta Complexa: Só funciona com consultas SQL simples que correspondem diretamente a uma tabela. Para consultas que usam JOINS, GROUP BY ou outras operações complexas, o SqlCommandBuilder não consegue gerar comandos automaticamente.
- Validação Limitada: Ele não verifica automaticamente regras de negócio ou restrições específicas de dados.

Em que situações pode ser adequado utilizar a representação dos dados em memória?

- Operações Offline: O sistema precisa funcionar sem acesso constante a uma base de dados ou à internet.
- Processamento Temporário de Dados: Os dados precisam ser manipulados ou calculados temporariamente sem a necessidade de salvá-los numa base de dados.
- Aplicações de Teste ou Desenvolvimento: Simular dados para testar funcionalidades sem depender de uma base de dados real.
- Minimizar o Acesso à Base de dados: Reduzir o número de conexões à base de dados para melhorar o desempenho e minimizar custos de infraestrutura.
- Manipulação em Massa: É necessário processar grandes volumes de dados antes de aplicá-los à base de dados.

- Cenários de Relatórios ou Dashboards: O sistema precisa criar relatórios ou dashboards interativos onde os dados são carregados uma vez e manipulados localmente.
 - Cache Temporário: Dados frequentemente usados são mantidos em memória para melhorar o desempenho e reduzir consultas à base de dados.
 - Trabalhar com Dados Heterogêneos: Dados provenientes de múltiplas fontes (bases de dados, ficheiros, APIs) precisam ser combinados e manipulados.
- Processamento de Dados Temporariamente Sensíveis: Dados que têm um ciclo de vida curto e não precisam ser persistidos.
- Redução de Sobrecarga na Base de dados: Sistemas de alto volume onde manter conexões abertas pode ser caro ou inviável.

Vantagens de Usar Representação em Memória

- Desempenho Rápido: Operações em memória são significativamente mais rápidas do que interações com a base de dados.
- Flexibilidade: Permite manipular dados dinamicamente sem restrições de um esquema fixo.
- Independência: Não depende de uma conexão constante com a base de dados.
- Redução de Carga na Base de Dados: Ideal para aplicações que precisam minimizar o impacto no servidor de base de dados.

Limitações e Cuidados

- Consumo de Memória: Manipular grandes volumes de dados em memória pode consumir muitos recursos.
- Persistência: Os dados armazenados em memória não são automaticamente salvos na base de dados ou no disco.
- Gestão de Conflitos: Alterações feitas em memória podem entrar em conflito com outras alterações quando sincronizadas com a base de dados.

Criação de Relações entre DataTables

É possível criar relações entre DataTables dentro de um DataSet no ADO.NET. As relações simulam associações de chave estrangeira entre tabelas, permitindo navegar entre os dados relacionados, como numa base de dados relacional.

Benefícios de Usar Relações no DataSet

- Simulação de Relacionamentos Relacionais: Permite trabalhar com dados relacionados em memória como numa base de dados.
- Navegação Simples: Relacionamentos permitem navegar diretamente entre tabelas pai e filho usando métodos como GetChildRows (para aceder a registos filhos) e GetParentRow (para aceder a registos pais).
- Organização de Dados: Útil para manipular dados desconectados, especialmente em cenários offline ou durante o processamento em memória.

Limitações

- Consumo de Memória: Trabalhar com grandes volumes de dados em memória pode ser caro em termos de recursos.
- Sincronização Manual: As alterações feitas nas tabelas relacionadas não são automaticamente sincronizadas com a base de dados.

É possível utilizar comandos sql entre datatables com relações definidas?

No ADO.NET, não é possível executar comandos SQL diretamente entre DataTables com relações definidas, pois as DataTables num DataSet são estruturas de dados em memória, e não possuem um mecanismo nativo para interpretar e executar SQL.

No entanto, podem simular-se operações como consultas SQL usando relações, filtros e expressões LINQ sobre as DataTables. Aqui estão as principais abordagens:

1. Usar Relações Definidas: Se existirem relações entre tabelas no DataSet, pode navegar-se pelos dados relacionados usando métodos como: GetChildRows e GetParentRow.
2. Filtrar com DataTable.Select: pode usar-se a função Select para aplicar filtros e condições semelhantes ao WHERE do SQL.
3. Usar LINQ para Consultas Mais Complexas: O LINQ (Language Integrated Query) permite realizar consultas poderosas e flexíveis entre DataTables.
4. Criar uma DataView para Operações em Tempo Real: Uma DataView permite criar uma visão filtrada ou ordenada de uma DataTable, o que é útil para simular operações de consulta.